ADVANCED PYTHON PROGRAMMING

UNIT III — MODULAR PROGRAMMING AND PACKAGE MANAGEMENT

Student Reference Notes

Definitions, installation and execution commands, runnable examples, testing, debugging, maintainable design, and a complete Student Marks CLI project.

How to use this document

- Read the definitions and examples in order.
- Type code into files using the exact filenames shown.
- Run commands from the project root unless noted otherwise.
- Run tests after changing code and investigate failures.

Commands primarily use Windows PowerShell. macOS/Linux alternatives are included where useful.

Contents

• 1. Modular programming fundamentals

• 2. Modules, imports, and the main guard

• 3. Packages and project structure

• 4. Standard library and third-party packages

• 5. pip and virtual environments

• 6. Testing with pytest and unittest

• 7. Debugging with pdb

• 8. Maintainable application design

• 9. Student Marks CLI mini-project

• 10. Troubleshooting

• 11. Revision checklist and questions

1. Modular programming fundamentals

Modular programming organizes a program into smaller, focused, reusable parts called modules. Each module should have a clear responsibility.

Module: a Python file containing definitions and statements, usually ending in .py. **Package:** a directory organizing related modules. **Import:** a statement that makes names from another module/package available. **Dependency:** code or a package another component relies on. **Separation of concerns:** keeping distinct responsibilities in distinct components.

Benefits include reuse, easier testing, simpler debugging, clearer organization, and safer changes.

Example: two modules

```
# calculator.py
def add(a, b):
    return a + b

# main.py
from calculator import add
print(add(10, 20))
```

Save both files in one folder and run `python main.py`. Output: 30.

2. Modules, imports, and the main guard

2.1 Common import forms

```
import math
print(math.sqrt(25))

from math import sqrt
print(sqrt(25))

import math as m
print(m.pi)
```

Prefer explicit imports. Avoid `from module import *` because it makes names less clear.

2.2 Import side effects

On its first import in a process, Python executes a module's top-level statements. Prompts, file writes, network calls, and demo calls at top level may run unexpectedly during import.

```
# Avoid this in calculator.py
def calculate_average(marks):
    return sum(marks) / len(marks)

print(calculate_average([70, 80, 90]))  # executes during import
```

2.3 The main guard

```
# main.py
def main():
    print("Program started")

if __name__ == "__main__":
```

```
    main()
```

Direct execution sets `__name__` to `'__main__'`. Importing the file does not automatically run the guarded CLI.

2.4 Execution commands

```
python main.py
py main.py
py -3.13 main.py
```

Use a version-specific command only if that Python interpreter is installed.

3. Packages and project structure

A package groups related modules and provides a namespace for imports.

```
student_marks/
    __init__.py
    calculator.py
    main.py
    tests/
        test_calculator.py
```

If the parent folder is on Python's import path:

```
from student_marks.calculator import calculate_average
```

If main.py and calculator.py are in the same directory:

```
from calculator import calculate_average
```

The correct import depends on project layout and the directory used to launch Python. Use a consistent layout and run from the intended project root.

```
student_marks/
    calculator.py
    main.py
    test_calculator.py
    README.md
    requirements.txt
```

• calculator.py: calculations.

• main.py: command-line workflow.

• test_calculator.py: automated tests.

• README.md: project and run instructions.

• requirements.txt: third-party dependencies.

4. Standard library and third-party packages

The **standard library** ships with Python. Examples: math, pathlib, json, csv, sqlite3, unittest, pdb. A **third-party package** is installed separately; pytest is one example. Use standard-library tools when they meet the need and add dependencies deliberately.

5. pip and virtual environments

5.1 pip commands

pip installs and manages third-party Python packages. Prefer `python -m pip` so installation is associated with the selected Python interpreter.

```
python -m pip --version
python -m pip install pytest
python -m pip show pytest
python -m pip list
python -m pip uninstall pytest
```

5.2 Virtual environments

A **virtual environment** isolates packages for one project from those used by other projects. This reduces version conflicts and helps make setup reproducible.

5.3 Windows PowerShell

```
# Run from the project folder
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python --version
```

```
python -m pip --version
python -m pip install pytest
```

For Command Prompt, use `.venv\Scripts\activate.bat`. Activation is optional if you run the environment interpreter directly: `.venv\Scripts\python.exe -m pytest -v`.

5.4 macOS/Linux

```
python3 -m venv .venv
source .venv/bin/activate
python --version
python -m pip install pytest
```

5.5 Save/reinstall dependencies and deactivate

```
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
deactivate
```

Share requirements.txt, not the .venv folder. The virtual environment directory is usually excluded from version control.

6. Testing with pytest and unittest

6.1 Testing definitions

Software testing: checking whether software behaves as expected. **Unit test:** checks a small unit, such as a function. **Assertion:** a condition that must hold for a test to pass. **Regression test:** helps prevent a fixed bug from returning. **Boundary test:** checks values at or near a limit. **Invalid-input test:** checks unacceptable input.

6.2 pytest commands

```
python -m pip install pytest
python -m pytest
python -m pytest -v
python -m pytest test_calculator.py -v
python -m pytest -k average -v
```

pytest commonly discovers files named `test_*.py` or `*_test.py` and functions beginning with `test_`.

6.3 Application code: calculator.py

```
def calculate_average(marks):
    if not marks:
        raise ValueError("Marks cannot be empty")
    return sum(marks) / len(marks)

def calculate_grade(average):
    if average >= 90:
        return "A"
    elif average >= 75:
        return "B"
    elif average >= 40:
        return "C"
    else:
        return "Fail"
```

6.4 Tests: test_calculator.py

```
import pytest
from calculator import calculate_average, calculate_grade

def test_calculate_average():
    assert calculate_average([70, 80, 90]) == 80

def test_empty_marks_raise_error():
    with pytest.raises(ValueError):
        calculate_average([])

def test_grade_a():
    assert calculate_grade(95) == "A"

def test_grade_boundary():
    assert calculate_grade(75) == "B"
```

Keep these files in the same project directory for these imports. Run `python -m pytest -v` from that directory.

6.5 Testing exceptions

```
import pytest

def test_empty_input():
    with pytest.raises(ValueError):
```

```
        calculate_average([])
```

Use `pytest.raises(ExpectedException)` when an exception is the expected behavior.

6.6 Testing CLI input/output

`monkeypatch` replaces interactive input with predictable values. `capsys` captures printed output.

```
# test_main.py
from main import main

def test_main_valid_input(monkeypatch, capsys):
    inputs = iter(["70", "80", "90"])
    monkeypatch.setattr(
        "builtins.input", lambda prompt: next(inputs)
    )
    main()
    captured = capsys.readouterr()
    assert "Average: 80.00" in captured.out
    assert "Grade: B" in captured.out
```

The import must match the project layout. Keep main() callable and protected by the main guard so importing main.py does not start the CLI.

6.7 Interpreting test results

• PASSED: all assertions held.

• FAILED: an assertion or expected behavior did not match.

• ERROR/collection error: pytest could not prepare or import a test/module.

• Traceback: sequence of calls leading to an exception.

6.8 unittest basics

`unittest` is built into Python. Tests commonly use a class derived from `unittest.TestCase`.

```
# test_unittest_example.py
import unittest
from calculator import calculate_average

class TestAverage(unittest.TestCase):
    def test_average(self):
        self.assertEqual(calculate_average([70, 80, 90]), 80)

    def test_empty_marks(self):
        with self.assertRaises(ValueError):
            calculate_average([])

if __name__ == "__main__":
    unittest.main()
```

```
python -m unittest -v
```

pytest provides concise assertions and fixtures; unittest is built into Python and uses test classes and assertion methods. Both are valid testing frameworks.

7. Debugging with pdb

Debugging is locating and correcting the cause of unexpected behavior. The built-in `pdb` debugger pauses execution so you can inspect values and control flow.

```
def total_prices(prices):
    total = 0
    for price in prices:
        breakpoint()
        total += price
    return total

print(total_prices([10, 20, 30]))
```

At the debugger prompt:

Command	Purpose
p expression	Print a value, e.g. p total
n	Next line; do not step into a call

s	Step into a function call
c	Continue to another breakpoint or program end
where	Display the current call stack
q	Quit the debugger
h	Show help

Debugging workflow: reproduce the issue → read the traceback → inspect values → step through execution → fix the cause → rerun the failing test and the full test suite.

8. Maintainable application design

Maintainable software can be understood, tested, debugged, and changed with reasonable effort.

8.1 Separation of concerns

In a marks application, input validation, calculation, reporting, and file export are separate responsibilities.

```
student_marks/
    calculator.py       # average and grade
    input_handler.py    # collect and validate input
    reporter.py         # format/display results
    exporter.py         # save results if needed
    main.py             # coordinate workflow
    tests/
        test_calculator.py
```

A small application does not need every module immediately. Add modules when they establish a useful boundary.

8.2 Pure functions and side effects

A **pure function** returns a result based only on its inputs and does not change external state; it is usually easy to test.

```
def calculate_average(marks):
    if not marks:
        raise ValueError("Marks cannot be empty")
    return sum(marks) / len(marks)
```

Input, printing, file writes, and network calls are side effects. Keeping them separate from calculation logic simplifies tests.

8.3 Practical design rules

• Give functions clear names and one primary responsibility.

• Validate external input at the application boundary.

• Raise meaningful exceptions for invalid arguments.

• Avoid unexplained global mutable state.

• Keep imports and dependencies explicit.

• Test normal behavior, boundaries, and invalid cases.

• Document how to install, run, and test the project.

9. Mini-project: Student Marks Management CLI

Goal: accept three marks, validate them, calculate the average, assign a grade, and display the result.

9.1 Requirements

• Accept three numeric marks.

• Each mark must be between 0 and 100 inclusive.

• Calculate the average.

• Grade: A for average ≥ 90; B for ≥ 75; C for ≥ 40; otherwise Fail.

• Handle non-numeric input and out-of-range marks.

• Keep calculations in a separate module and test them.

9.2 calculator.py

```
def calculate_average(marks):
    """Return the average of a non-empty collection of marks."""
    if not marks:
        raise ValueError("Marks cannot be empty")
    return sum(marks) / len(marks)


def calculate_grade(average):
    """Return the grade based on the average."""
    if average >= 90:
        return "A"
    elif average >= 75:
        return "B"
    elif average >= 40:
        return "C"
    else:
        return "Fail"
```

9.3 main.py

```
from calculator import calculate_average, calculate_grade

def main():
    try:
        marks = []
        for i in range(3):
            mark = float(input(f"Enter mark {i + 1}: "))
            if not 0 <= mark <= 100:
                raise ValueError("Marks must be between 0 and 100")
            marks.append(mark)

        average = calculate_average(marks)
        grade = calculate_grade(average)
        print(f"Average: {average:.2f}")
        print(f"Grade: {grade}")

    except ValueError as error:
        print(f"Invalid input: {error}")

if __name__ == "__main__":
    main()
```

9.4 test_calculator.py

```
import pytest
from calculator import calculate_average, calculate_grade

def test_calculate_average():
    assert calculate_average([70, 80, 90]) == 80

def test_empty_marks_raise_error():
    with pytest.raises(ValueError):
        calculate_average([])

def test_calculate_grade_a():
    assert calculate_grade(95) == "A"

def test_calculate_grade_boundary():
    assert calculate_grade(75) == "B"
```

9.5 Create, install, test, and run

```
mkdir student_marks
cd student_marks

# Windows PowerShell
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install pytest

# Create calculator.py, main.py, test_calculator.py
# Paste the code above into the matching files.

python -m pytest -v
python main.py
```

9.6 Successful execution

```
Enter mark 1: 70
Enter mark 2: 80
Enter mark 3: 90
Average: 80.00
```

```
Grade: B
```

9.7 Invalid mark

```
Enter mark 1: 70
Enter mark 2: 80
Enter mark 3: 120
Invalid input: Marks must be between 0 and 100
```

Entering abc where a number is expected raises ValueError during float conversion and is handled by the same exception handler.

9.8 Optional automated CLI test

```
# test_main.py
from main import main

def test_main_valid_input(monkeypatch, capsys):
    inputs = iter(["70", "80", "90"])
    monkeypatch.setattr(
        "builtins.input", lambda prompt: next(inputs)
    )
    main()
    captured = capsys.readouterr()
    assert "Average: 80.00" in captured.out
    assert "Grade: B" in captured.out
```

Run all tests with `python -m pytest -v`. This import assumes test_main.py and main.py are in the same directory.

10. Troubleshooting

ModuleNotFoundError

Python cannot find the module. Check spelling, filename, folder structure, import statement, current working directory, and the command used to launch the program.

pytest collection/import error

Pytest could not import a test or application module. Check import paths and top-level statements that execute and raise errors during import.

pytest finds no tests

Use names such as test_calculator.py and test functions beginning with test_. Run pytest from the correct directory.

pytest is not installed

```
python -m pip install pytest
python -m pytest -v
```

If installation appears successful but import fails, check that the IDE and terminal use the same interpreter and virtual environment.

PowerShell activation problem

Check that .venv\Scripts\Activate.ps1 exists and use the correct terminal. Alternatively run .venv\Scripts\python.exe -m pytest -v without activation.

Incorrect average or grade

Check input values, arithmetic, comparison order, and grade thresholds. Add a test reproducing the wrong result before changing code.

11. Revision checklist

- ■ I can define module, package, import, dependency, and separation of concerns.
- ■ I can import a function from one Python file into another.
- ■ I can explain top-level execution during import and use the main guard.
- ■ I can create/activate a venv and install packages with python -m pip.
- ■ I can run pytest and interpret PASSED, FAILED, and collection errors.
- ■ I can test normal input, empty input, boundaries, and exceptions.
- ■ I can explain the basic structure of unittest tests.

■ I can use breakpoint(), p, n, s, c, where, and q in pdb.

■ I can separate calculation logic from input/output.

■ I can run the Student Marks CLI and its automated tests.

Practice and interview questions

• 1. What is a module? How is it different from a package?

• 2. What happens when a Python module is imported?

• 3. Why use if __name__ == '__main__'?

• 4. Compare the standard library with a third-party package.

• 5. Why use a virtual environment for each project?

• 6. What does python -m pip install pytest do?

• 7. What is a unit test? What is a regression test?

• 8. How does pytest.raises() verify exception behavior?

• 9. How do monkeypatch and capsys help test a CLI?

• 10. Compare pytest and unittest.

• 11. How do p, n, s, and where help in pdb?

• 12. Why separate calculations from input/output?

• 13. Write tests for grade boundaries at 40, 75, and 90.

• 14. How does requirements.txt help recreate an environment?

• 15. How would you investigate ModuleNotFoundError?

End of Unit III Student Reference